EXPERIMENT 4

AIM: To create an interactive form in Flutter using the **Form widget** and input fields with validation.

OBJECTIVE: To understand how to create a form, accept user input, validate the entered data, and perform an action when the form is submitted.

COURSE OUTCOME: Design and implement user interfaces using Flutter widgets, layouts, and animations, ensuring responsiveness and interactivity

THEORY: The **Form widget** in Flutter is used to group and manage multiple input fields. It works with widgets such as TextFormField to collect information from users. Form validation helps ensure that the entered data is correct before submission. A GlobalKey<FormState> is commonly used to access and validate the form. The validator property of TextFormField checks the input and displays an error message when the entered data is invalid.

Software Requirements: -

- Flutter SDK
- Dart
- Android Studio / DartPad
- Web browser

CODE:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const GovtFormApp());
}

class GovtFormApp extends
StatelessWidget {
  const GovtFormApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Government Service
Application Portal',
      debugShowCheckedModeBanner:
false,
      theme: ThemeData(
        colorScheme:
ColorScheme.fromSeed(
          seedColor: const
Color(0xFF0F3D3E),
          brightness: Brightness.light,
        ),
        scaffoldBackgroundColor: const
Color(0xFFF4F6F8),
        useMaterial3: true,
      ),
      home: const GovtFormScreen(),
    );
  }
}

class GovtFormScreen extends
StatefulWidget {
  const GovtFormScreen({super.key});

  @override
  State<GovtFormScreen> createState()
=> _GovtFormScreenState();
}

class _GovtFormScreenState extends
State<GovtFormScreen> {
  final _formKey =
GlobalKey<FormState>();

  // Form Controllers
  final _nameController =
TextEditingController();
  final _idController =
TextEditingController();
  final _dobController =
TextEditingController();
  final _emailController =
TextEditingController();
  final _phoneController =
TextEditingController();
  final _addressController =
TextEditingController();
  final _pincodeController =
TextEditingController();

  // Selection states
  String? _gender = 'Male';
  String? _serviceType = 'Passport
```

```
Renewal & Issue';
  String? _state = 'California / State
Portal';
  bool _declarationAccepted = false;
  bool _isSubmitted = false;
  String _applicationId = '';

  final List<String> _genderOptions =
['Male', 'Female', 'Other', 'Prefer not to
say'];

  final List<String> _serviceOptions =
[
    'Passport Renewal & Issue',
    'National Identity Card
(Aadhaar/SSN)',
    'Income & Caste Certificate',
    'Driving License & Vehicle Reg',
    'Citizen Welfare Grant Scheme',
  ];

  final List<String> _stateOptions = [
    'California / State Portal',
    'New York / Central Region',
    'Texas / Southern District',
    'Delhi / National Capital Territory',
    'Maharashtra / Western Zone',
  ];

  @override
  void dispose() {
    _nameController.dispose();
    _idController.dispose();
    _dobController.dispose();
    _emailController.dispose();
    _phoneController.dispose();
    _addressController.dispose();
    _pincodeController.dispose();
    super.dispose();
  }

  void _submitForm() {
    if
(_formKey.currentState!.validate()) {
      if (!_declarationAccepted) {
        ScaffoldMessenger.of(context).sh
owSnackBar(
          const SnackBar(
            content: Text('Please accept the
official declaration to proceed.'),
            backgroundColor:
Colors.redAccent,
            behavior:
SnackBarBehavior.floating,
          ),
        );
        return;
      }

      setState(() {
        _applicationId = 'GOVT-2026-
${(10000 +
(DateTime.now().millisecondsSinceEp
och % 89999))}';
        _isSubmitted = true;
      });

      ScaffoldMessenger.of(context).sho
wSnackBar(
        SnackBar(
          content: Text('Application
Submitted Successfully! ID:
$_applicationId'),
          backgroundColor: const
Color(0xFF0F3D3E),
          behavior:
SnackBarBehavior.floating,
        ),
      );
    }
  }

  void _resetForm() {
    _formKey.currentState?.reset();
    _nameController.clear();
    _idController.clear();
    _dobController.clear();
    _emailController.clear();
    _phoneController.clear();
    _addressController.clear();
    _pincodeController.clear();
    setState(() {
      _gender = 'Male';
      _serviceType =
_serviceOptions.first;
      _state = _stateOptions.first;
      _declarationAccepted = false;
      _isSubmitted = false;
      _applicationId = '';
    });
  }

  void _triggerPrint() {
    ScaffoldMessenger.of(context).show
SnackBar(
      const SnackBar(
        content: Row(
          children: [
            Icon(Icons.print_rounded,
color: Colors.white),
            SizedBox(width: 12),
            Expanded(child: Text('Opening
Print Preview & Exporting Official
Document...')),
          ],
        ),
        behavior:
SnackBarBehavior.floating,
        backgroundColor:
Color(0xFF0F3D3E),
```

```
    ),
   );
  }

  Future<void> _selectDateOfBirth()
async {
   final DateTime? picked = await
showDatePicker(
    context: context,
    initialDate: DateTime(2000, 1, 1),
    firstDate: DateTime(1920),
    lastDate: DateTime.now(),
   );
   if (picked != null) {
    setState(() {
     _dobController.text =
"${picked.day.toString().padLeft(2,
'0')}/${picked.month.toString().padLef
t(2, '0')}/${picked.year}";
    });
   }
  }

  @override
  Widget build(BuildContext context) {
   return Scaffold(
    appBar: AppBar(
     backgroundColor: const
Color(0xFF0F3D3E),
     foregroundColor: Colors.white,
     elevation: 2,
     title: const Row(
      children: [
       Icon(Icons.account_balance,
size: 28),
       SizedBox(width: 12),
       Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
         Text(
          'NATIONAL CITIZEN
PORTAL',
          style: TextStyle(fontSize:
16, fontWeight: FontWeight.bold,
letterSpacing: 1.1),
         ),
         Text(
          'Government Services
Official Application Form',
          style: TextStyle(fontSize:
11, color: Colors.white70),
         ),
        ],
       ),
      ],
     ),
     actions: [
      if (_isSubmitted)
       IconButton(
        icon: const
Icon(Icons.print_rounded),
        tooltip: 'Print Application
Receipt',
        onPressed: _triggerPrint,
       ),
      IconButton(
       icon: const
Icon(Icons.refresh_rounded),
       tooltip: 'Reset Form',
       onPressed: _resetForm,
      ),
      const SizedBox(width: 8),
     ],
    ),
    body: SingleChildScrollView(
     padding: const
EdgeInsets.symmetric(horizontal: 20,
vertical: 24),
     child: Center(
      child: ConstrainedBox(
       constraints: const
BoxConstraints(maxWidth: 800),
       child: _isSubmitted ?
_buildSubmittedReceiptView() :
_buildFormView(),
      ),
     ),
    ),
   );
  }

  // --- FORM INPUT VIEW ---
  Widget _buildFormView() {
   return Card(
    elevation: 4,
    shape:
RoundedRectangleBorder(borderRadiu
s: BorderRadius.circular(16)),
    child: Padding(
     padding: const
EdgeInsets.all(28.0),
     child: Form(
      key: _formKey,
      child: Column(
       crossAxisAlignment:
CrossAxisAlignment.start,
       children: [
        // Header Badge
        Container(
         padding: const
EdgeInsets.all(16),
         decoration: BoxDecoration(
          color: const
Color(0xFF0F3D3E).withValues(alpha
: 0.08),
          borderRadius:
BorderRadius.circular(12),
          border: Border.all(color:
const
```

```
Color(0xFF0F3D3E).withValues(alpha
: 0.2)),
            ),
            child: const Row(
              children: [
                Icon(Icons.verified_user_r
ounded, color: Color(0xFF0F3D3E),
size: 36),
                SizedBox(width: 16),
                Expanded(
                  child: Column(
                    crossAxisAlignment:
CrossAxisAlignment.start,
                    children: [
                      Text(
                        'OFFICIAL
APPLICATION FORM 2026',
                        style: TextStyle(
                          fontWeight:
FontWeight.bold,
                          fontSize: 16,
                          color:
Color(0xFF0F3D3E),
                        ),
                      ),
                      SizedBox(height: 4),
                      Text(
                        'Please fill in all
mandatory entries accurately as per
official government records.',
                        style:
TextStyle(fontSize: 12, color:
Colors.black87),
                      ),
                    ],
                  ),
                ),
              ],
            ),
          ),
          const SizedBox(height: 24),

          // SECTION 1: Service
Selection
          _buildSectionTitle('1. Service
& Jurisdiction Details'),
          const SizedBox(height: 12),
          DropdownButtonFormField<S
tring>(
            initialValue: _serviceType,
            decoration:
_inputDecoration('Service Requested
*', Icons.room_service_rounded),
            items: _serviceOptions
                .map((s) =>
DropdownMenuItem(value: s, child:
Text(s)))
                .toList(),
            onChanged: (val) =>
setState(() => _serviceType = val),
            validator: (val) => val ==
null ? 'Please select a service' : null,
          ),
          const SizedBox(height: 16),
          DropdownButtonFormField<S
tring>(
            initialValue: _state,
            decoration:
_inputDecoration('State / Regional
Jurisdiction *',
Icons.location_city_rounded),
            items: _stateOptions
                .map((st) =>
DropdownMenuItem(value: st, child:
Text(st)))
                .toList(),
            onChanged: (val) =>
setState(() => _state = val),
            validator: (val) => val ==
null ? 'Please select your state' : null,
          ),
          const SizedBox(height: 24),

          // SECTION 2: Personal
Information
          _buildSectionTitle('2.
Personal Applicant Information'),
          const SizedBox(height: 12),
          TextFormField(
            controller: _nameController,
            decoration:
_inputDecoration('Full Legal Name (as
per Govt ID) *',
Icons.person_rounded),
            validator: (val) {
              if (val == null ||
val.trim().isEmpty) return 'Full name is
required';
              if (val.trim().length < 3)
return 'Name must be at least 3
characters';
              return null;
            },
          ),
          const SizedBox(height: 16),
          Row(
            children: [
              Expanded(
                child: TextFormField(
                  controller: _idController,
                  decoration:
_inputDecoration('Citizen ID / SSN /
Aadhaar No *', Icons.badge_rounded),
                  validator: (val) {
                    if (val == null ||
val.trim().isEmpty) return 'Govt ID
number is required';
                    if (val.trim().length < 6)
return 'Enter a valid ID number';
                    return null;
```

```
                  },
                ),
              ),
              const SizedBox(width: 16),
              Expanded(
                child: TextFormField(
                  controller:
_dobController,
                  readOnly: true,
                  onTap:
_selectDateOfBirth,
                  decoration:
_inputDecoration('Date of Birth
(DD/MM/YYYY) *',
Icons.calendar_today_rounded),
                  validator: (val) => (val
== null || val.isEmpty) ? 'Select date of
birth' : null,
                ),
              ),
            ],
          ),
          const SizedBox(height: 16),
          DropdownButtonFormField<S
tring>(
            initialValue: _gender,
            decoration:
_inputDecoration('Gender *',
Icons.people_rounded),
            items: _genderOptions
                .map((g) =>
DropdownMenuItem(value: g, child:
Text(g)))
                .toList(),
            onChanged: (val) =>
setState(() => _gender = val),
          ),
          const SizedBox(height: 24),

          // SECTION 3: Contact &
Address
          _buildSectionTitle('3. Contact
& Communication Address'),
          const SizedBox(height: 12),
          Row(
            children: [
              Expanded(
                child: TextFormField(
                  controller:
_emailController,
                  keyboardType:
TextInputType.emailAddress,
                  decoration:
_inputDecoration('Email Address *',
Icons.email_rounded),
                  validator: (val) {
                    if (val == null ||
val.trim().isEmpty) return 'Email
address is required';
                    if (!RegExp(r'^[\w-
\.]+@([\w-]+\.)+[\w-
]{2,4}$').hasMatch(val)) {
                      return 'Enter a valid
email address';
                    }
                    return null;
                  },
                ),
              ),
              const SizedBox(width: 16),
              Expanded(
                child: TextFormField(
                  controller:
_phoneController,
                  keyboardType:
TextInputType.phone,
                  decoration:
_inputDecoration('Mobile Number *',
Icons.phone_rounded),
                  validator: (val) {
                    if (val == null ||
val.trim().isEmpty) return 'Phone
number is required';
                    if (val.trim().length <
10) return 'Enter 10-digit mobile
number';
                    return null;
                  },
                ),
              ),
            ],
          ),
          const SizedBox(height: 16),
          TextFormField(
            controller:
_addressController,
            maxLines: 2,
            decoration:
_inputDecoration('Permanent
Residential Address *',
Icons.home_rounded),
            validator: (val) => (val ==
null || val.trim().isEmpty) ? 'Residential
address is required' : null,
          ),
          const SizedBox(height: 16),
          TextFormField(
            controller:
_pincodeController,
            keyboardType:
TextInputType.number,
            decoration:
_inputDecoration('Pincode / Postal Zip
Code *',
Icons.markunread_mailbox_rounded),
            validator: (val) {
              if (val == null ||
val.trim().isEmpty) return 'Pincode is
required';
              if (val.trim().length < 5)
```

```
return 'Enter a valid postal code';
                return null;
              },
            ),
            const SizedBox(height: 24),

            // SECTION 4: Declaration
Checkbox
            _buildSectionTitle('4. Official
Legal Declaration'),
            const SizedBox(height: 8),
            FormField<bool>(
              initialValue:
_declarationAccepted,
              validator: (val) =>
_declarationAccepted ? null : 'You
must accept the declaration',
              builder: (state) {
                return Column(
                  crossAxisAlignment:
CrossAxisAlignment.start,
                  children: [
                    CheckboxListTile(
                      contentPadding:
EdgeInsets.zero,
                      title: const Text(
                        'I hereby declare that
all provided statements and document
details are true, correct, and complete
to the best of my knowledge.',
                        style:
TextStyle(fontSize: 13, height: 1.4),
                      ),
                      value:
_declarationAccepted,
                      activeColor: const
Color(0xFF0F3D3E),
                      onChanged: (val) {
                        setState(() {
                          _declarationAccepted
= val ?? false;
                        });
                        state.didChange(val);
                      },
                    ),
                    if (state.hasError)
                      Padding(
                        padding: const
EdgeInsets.only(left: 12),
                        child: Text(
                          state.errorText!,
                          style: const
TextStyle(color: Colors.redAccent,
fontSize: 12),
                        ),
                      ),
                  ],
                );
              },
            ),
            const SizedBox(height: 32),

            // SUBMIT & RESET
BUTTONS
            Row(
              mainAxisAlignment:
MainAxisAlignment.end,
              children: [
                OutlinedButton.icon(
                  onPressed: _resetForm,
                  icon: const
Icon(Icons.clear_all_rounded),
                  label: const Text('Reset
Form'),
                  style:
OutlinedButton.styleFrom(
                    padding: const
EdgeInsets.symmetric(horizontal: 24,
vertical: 16),
                    shape:
RoundedRectangleBorder(borderRadiu
s: BorderRadius.circular(12)),
                  ),
                ),
                const SizedBox(width: 16),
                ElevatedButton.icon(
                  onPressed: _submitForm,
                  icon: const
Icon(Icons.send_rounded),
                  label: const Text('Submit
Application'),
                  style:
ElevatedButton.styleFrom(
                    backgroundColor: const
Color(0xFF0F3D3E),
                    foregroundColor:
Colors.white,
                    padding: const
EdgeInsets.symmetric(horizontal: 32,
vertical: 16),
                    shape:
RoundedRectangleBorder(borderRadiu
s: BorderRadius.circular(12)),
                    elevation: 2,
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    ),
  );
 }

  // --- SUBMITTED RECEIPT &
PRINT PREVIEW VIEW ---
  Widget
_buildSubmittedReceiptView() {
    return Card(
```

```
      elevation: 6,
      shape:
RoundedRectangleBorder(borderRadiu
s: BorderRadius.circular(16)),
      child: Padding(
        padding: const
EdgeInsets.all(32.0),
        child: Column(
          crossAxisAlignment:
CrossAxisAlignment.start,
          children: [
            // Official Header Banner for
Print
            Center(
              child: Column(
                children: [
                  const
Icon(Icons.account_balance, size: 48,
color: Color(0xFF0F3D3E)),
                  const SizedBox(height: 8),
                  const Text(
                    'GOVERNMENT OF
NATIONAL SERVICES',
                    style:
TextStyle(fontWeight:
FontWeight.bold, fontSize: 18,
letterSpacing: 1.2),
                  ),
                  const Text(
                    'OFFICIAL CITIZEN
APPLICATION
ACKNOWLEDGEMENT RECEIPT',
                    style: TextStyle(fontSize:
12, color: Colors.grey, fontWeight:
FontWeight.bold),
                  ),
                  const SizedBox(height: 12),
                  Chip(
                    avatar: const
Icon(Icons.check_circle, color:
Colors.green),
                    label: Text(
                      'STATUS:
APPLICATION SUBMITTED • REF:
$_applicationId',
                      style: const
TextStyle(fontWeight:
FontWeight.bold, fontSize: 12),
                    ),
                    backgroundColor:
Colors.green.withValues(alpha: 0.1),
                  ),
                ],
              ),
            ),
            const Divider(height: 36,
thickness: 1.5),

            // Entries Summary Table
            const Text(
              'APPLICATION ENTRY
SUMMARY',
              style: TextStyle(fontSize: 14,
fontWeight: FontWeight.bold, color:
Color(0xFF0F3D3E)),
            ),
            const SizedBox(height: 16),
            _buildReceiptRow('Application
Reference ID', _applicationId, isBold:
true),
            _buildReceiptRow('Service
Requested', _serviceType ?? '-'),
            _buildReceiptRow('State /
Regional Jurisdiction', _state ?? '-'),
            const Divider(height: 24),
            _buildReceiptRow('Applicant
Full Name', _nameController.text),
            _buildReceiptRow('Govt ID /
SSN / Aadhaar', _idController.text),
            _buildReceiptRow('Date of
Birth', _dobController.text),
            _buildReceiptRow('Gender',
_gender ?? '-'),
            const Divider(height: 24),
            _buildReceiptRow('Email
Address', _emailController.text),
            _buildReceiptRow('Mobile
Phone Number',
_phoneController.text),
            _buildReceiptRow('Residential
Address', _addressController.text),
            _buildReceiptRow('Postal Code
/ Pincode', _pincodeController.text),
            _buildReceiptRow('Submission
Timestamp',
DateTime.now().toString().substring(0,
19)),
            _buildReceiptRow('Declaration
Status', 'VERIFIED & SIGNED
ELECTRONICALLY'),
            const SizedBox(height: 32),

            // Action Buttons (Print / Edit)
            Row(
              mainAxisAlignment:
MainAxisAlignment.spaceBetween,
              children: [
                OutlinedButton.icon(
                  onPressed: () {
                    setState(() {
                      _isSubmitted = false;
                    });
                  },
                  icon: const
Icon(Icons.edit_note_rounded),
                  label: const Text('Edit
Entries'),
                  style:
OutlinedButton.styleFrom(
                    padding: const
```

```
EdgeInsets.symmetric(horizontal: 20,
vertical: 14),
                ),
              ),
              Row(
                children: [
                  ElevatedButton.icon(
                    onPressed: _triggerPrint,
                    icon: const
Icon(Icons.print_rounded),
                    label: const Text('Print
Application Form / Save PDF'),
                    style:
ElevatedButton.styleFrom(
                      backgroundColor: const
Color(0xFF0F3D3E),
                      foregroundColor:
Colors.white,
                      padding: const
EdgeInsets.symmetric(horizontal: 28,
vertical: 16),
                      shape:
RoundedRectangleBorder(borderRadiu
s: BorderRadius.circular(12)),
                    ),
                  ),
                ],
              ),
            ],
          ),
        ],
      ),
    ),
  );
 }

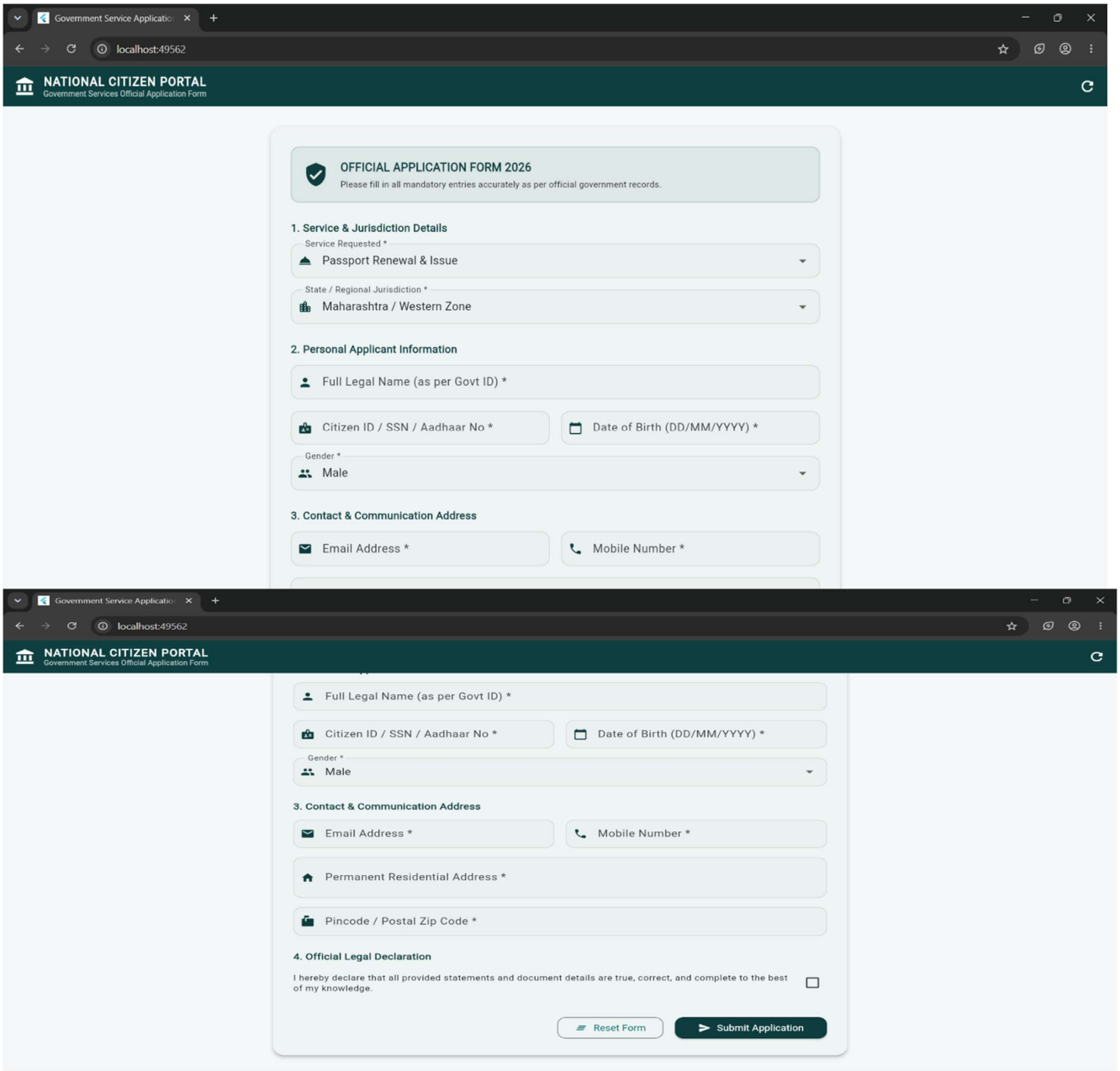
  Widget _buildReceiptRow(String
label, String value, {bool isBold =
false}) {
    return Padding(
      padding: const
EdgeInsets.symmetric(vertical: 6.0),
      child: Row(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          SizedBox(
            width: 220,
            child: Text(
              label,
              style: const
TextStyle(fontWeight:
FontWeight.w600, color:
Colors.black54, fontSize: 13),
            ),
          ),
          const Text(' :  '),
          Expanded(
            child: Text(
              value,
              style: TextStyle(
                fontWeight: isBold ?
FontWeight.bold : FontWeight.w500,
                fontSize: 13,
                color: isBold ? const
Color(0xFF0F3D3E) : Colors.black87,
              ),
            ),
          ),
        ],
      ),
    );
  }

  Widget _buildSectionTitle(String
title) {
    return Text(
      title,
      style: const TextStyle(
        fontSize: 15,
        fontWeight: FontWeight.bold,
        color: Color(0xFF0F3D3E),
      ),
    );
  }

  InputDecoration
_inputDecoration(String label,
IconData icon) {
    return InputDecoration(
      labelText: label,
      prefixIcon: Icon(icon, color: const
Color(0xFF0F3D3E), size: 20),
      filled: true,
      fillColor:
Colors.grey.withValues(alpha: 0.05),
      border: OutlineInputBorder(
        borderRadius:
BorderRadius.circular(12),
        borderSide: BorderSide(color:
Colors.grey.withValues(alpha: 0.3)),
      ),
      enabledBorder:
OutlineInputBorder(
        borderRadius:
BorderRadius.circular(12),
        borderSide: BorderSide(color:
Colors.grey.withValues(alpha: 0.3)),
      ),
      focusedBorder:
OutlineInputBorder(
        borderRadius:
BorderRadius.circular(12),
        borderSide: const
BorderSide(color:
Color(0xFF0F3D3E), width: 2),
      ),
      contentPadding: const
EdgeInsets.symmetric(horizontal: 16,
vertical: 14),
```

```
    );                                                    }
  }
```

OUTPUT:



CONCLUSION: The interactive Flutter form was successfully created using the Form and TextFormField widgets, with input validation and form submission functionality. The practical demonstrated how Flutter's Form widget and validation features can be used to create an interactive and user-friendly form for collecting and validating user input.